DIABETES PREDICTION — CAPSTONE PROJECT

Importing the Required Libraries

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

Load dataset

In [2]:
```
df = pd.read_csv(r"C:\Users\HI\Desktop\DataScience\Diabetes Prediction\diabetes.csv")
df.head()
```

Out[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33

Printing the shape, info, statastical analysis of dadaset

In [3]:
```
print("=" * 100)
print("Shape :", df.shape)
print("=" * 100)
print(df.info())
print("=" * 100)
print(df.describe())
print("=" * 100)
```

```
==========================================================================================
======
Shape : (768, 9)
==========================================================================================
======
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
==========================================================================================
======
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
==========================================================================================
======
```

Selecting X Features Columns

In [4]:
```
X = df.drop('Outcome', axis=1)
X
```

Out[4]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33
...	...	...	...	...	...	...	...	.
763	10	101	76	48	180	32.9	0.171	63
764	2	122	70	27	0	36.8	0.340	27
765	5	121	72	23	112	26.2	0.245	30
766	1	126	60	0	0	30.1	0.349	47
767	1	93	70	31	0	30.4	0.315	23

768 rows × 8 columns

Selecting y labels Columns

In [5]:

```
y = df['Outcome']
print(y.value_counts())
print(y.value_counts(normalize=True).round(3))
print("=" * 50)
print(y)
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
Outcome
0    0.651
1    0.349
Name: proportion, dtype: float64
==================================================
0      1
1      0
2      1
3      0
4      1
      ..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

Printing the Shape of X-Features and Y-Labels

In [6]:

```
print ("X : ", X.shape)
print ("y : ", y.shape)
```

```
X :  (768, 8)
y :  (768,)
```

HANDLE MISSING VALUES

Split Data for Training and Testing Process

Some Columns like insulin and skin thickness are having a value as 0, thats not a Correct value. they will generate outliers.

In [7]:
```
# Step 1: Split first
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 2: THEN replace zeros — separately on train and test
cols_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
X_train = X_train.copy()
X_test = X_test.copy()
X_train[cols_with_zeros] = X_train[cols_with_zeros].replace(0, np.nan)
X_test[cols_with_zeros] = X_test[cols_with_zeros].replace(0, np.nan)
print("\nTraining size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (614, 8)
Testing size: (154, 8)
```

BUILDING PIPELINES

Using one by one Algorithms!

In [8]:
```
# Applying Logistic algorithm with pipeline
pipe_lr = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # median bcz outliers!
    ('scaler',  StandardScaler()),                  # needed for LR!
    ('model',   LogisticRegression())
])
```

In [9]:
```
# Applying naive bayes algorithm with Pipeline
pipe_nb = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('model',   GaussianNB())                       # GaussianNB for numerical data!
])
```

In [10]:
```
pipe_dt = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('model', DecisionTreeClassifier(random_state=42))
])

param_grid = {'model__max_depth': [2, 3, 4, 5, 6, 7, 8]}

grid = GridSearchCV(pipe_dt, param_grid, cv=5, scoring='recall')
grid.fit(X_train, y_train)

print("Best max_depth:", grid.best_params_)
print("Best Recall:", round(grid.best_score_, 3))

best_pipe_dt = grid.best_estimator_
```

```
Best max_depth: {'model__max_depth': 7}
Best Recall: 0.686
```

TRAIN & CROSS VALIDATE

Training the Dataset to Multiple Algorithms

Also Providing Cross Validation Results

In [11]:
```
pipelines = {
    'Logistic Regression': pipe_lr,
    'Naive Bayes':         pipe_nb,
    'Decision Tree (Tuned)': best_pipe_dt
}

print("\n" + "="*50)
print("CROSS VALIDATION RESULTS (cv=5)")
print("="*50)

for name, pipe in pipelines.items():
    scores = cross_val_score(pipe, X_train, y_train, cv=5)
    print(f"{name:25} → Mean: {scores.mean():.2f} | Std: {scores.std():.2f}")
```

```
==================================================
CROSS VALIDATION RESULTS (cv=5)
==================================================
Logistic Regression       → Mean: 0.77 | Std: 0.03
Naive Bayes               → Mean: 0.75 | Std: 0.02
Decision Tree (Tuned)     → Mean: 0.72 | Std: 0.04
```

Evaluation value of Test data on EACH MODEL

In [12]:
```
print("\n" + "="*50)
print("DETAILED EVALUATION ON TEST DATA")
print("="*50)

for name, pipe in pipelines.items():
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    print(f"\n{'-'*40}")
    print(f"  {name}")
    print(f"{'-'*40}")
    print(confusion_matrix(y_test, y_pred))
    print(classification_report(y_test, y_pred))
```

```
==================================================
DETAILED EVALUATION ON TEST DATA
==================================================

────────────────────────────────────────
  Logistic Regression
────────────────────────────────────────
[[82 17]
 [21 34]]
              precision    recall  f1-score   support

           0       0.80      0.83      0.81        99
           1       0.67      0.62      0.64        55

    accuracy                           0.75       154
   macro avg       0.73      0.72      0.73       154
weighted avg       0.75      0.75      0.75       154


────────────────────────────────────────
  Naive Bayes
────────────────────────────────────────
[[79 20]
 [17 38]]
              precision    recall  f1-score   support

           0       0.82      0.80      0.81        99
           1       0.66      0.69      0.67        55

    accuracy                           0.76       154
   macro avg       0.74      0.74      0.74       154
weighted avg       0.76      0.76      0.76       154


────────────────────────────────────────
  Decision Tree (Tuned)
────────────────────────────────────────
[[71 28]
 [13 42]]
              precision    recall  f1-score   support

           0       0.85      0.72      0.78        99
           1       0.60      0.76      0.67        55

    accuracy                           0.73       154
   macro avg       0.72      0.74      0.72       154
weighted avg       0.76      0.73      0.74       154
```

Picking the best model out of Three

Focusing on metric → RECALL bcz (missing diabetic = dangerous!)

In [13]:
```python
best_model = best_pipe_dt
best_model.fit(X_train, y_train)

print("\nFinal Model Accuracy:", best_model.score(X_test, y_test))
print("\nFinal Classification Report:")
print(classification_report(y_test, best_model.predict(X_test)))
print("="* 100)
```

```
Final Model Accuracy: 0.7337662337662337

Final Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.72      0.78        99
           1       0.60      0.76      0.67        55

    accuracy                           0.73       154
   macro avg       0.72      0.74      0.72       154
weighted avg       0.76      0.73      0.74       154

==========================================================================================
======
```

Making Final decision and Summary

After comparing all 3 algorithms on the Diabetes Prediction dataset:

1. Logistic Regression gave highest Cross Validation accuracy (77%) but **lowest Recall (0.62)** for detecting diabetes
2. Naive Bayes was most consistent (Std=0.02) with **decent Recall (0.69)**
3. Decision Tree gave lowest overall accuracy (72%) but **HIGHEST Recall (0.76)**

I selected Decision Tree as my final model because:

- Missing a diabetic patient is more dangerous than a false alarm
- Recall matters most here — Decision Tree catches 76% of actual diabetics
- That's 8 more patients saved compared to Logistic Regression!"

In [ ]: